

Offline Forensic Dual-Pipeline Automator

Air-Gapped Workstation Integration Guide (PA10 & Axiom)

1. Offline Air-Gapped Deployment Architecture

In highly secure environments, forensic workstations run completely isolated from the internet (air-gapped). Shifting from Python to a native Windows **PowerShell + WPF (Windows Presentation Foundation)** architecture eliminates the need to install runtime engines, download pip packages, or verify online repository certificates. The script operates entirely using embedded local assemblies.

2. Air-Gapped System Requirements Matrix

Component	Offline Air-Gapped Requirement	Forensic Purpose
Operating System	Windows 10 / 11 Enterprise (64-bit) fully self-contained.	Executes localized background runspaces without WAN checkbacks.
Licensing Layer	Local USB Security Dongle (HASP) or a mapped offline local network license server.	Validates tool authentication completely offline during loops.
Storage Array	Drive C: OS/Apps (SSD) Drive D: Cold Evidence Storage Drive E & F: Isolated high-speed NVMe scratch spaces.	Prevents multi-engine I/O deadlocks by segregating file processing tracks.
Runtime Engine	Native PowerShell 5.1+ (Built into Windows). No Python or external dependencies required.	Constructs local WPF GUI panels and mirrors multi-thread workloads natively.

3. Complete Native PowerShell GUI Source Code

Save the code block below as **ForensicPipeline_Offline.ps1** on your workstation. Since it uses pure WPF and native .NET hooks, it will immediately launch an interactive panel when executed on any secure system.

```
Add-Type -AssemblyName PresentationFramework, System.Windows.Forms

# 1. Define the WPF User Interface XML (XAML)
[xml]$XAML = @"
    xmlns:x="http://schemas.microsoft.com/winfx/2000/xaml"
    Title="Offline Forensic Dual-Pipeline Automator (PowerShell Native)" Height="550" Width="650"
    Background="#F0F0F0">
```

```
FontFamily="Consolas" VerticalScrollBarVisibility="Auto" AcceptsReturn="True"/>
```

```
"@
```

```
$Reader = (New-Object System.Xml.XmlNodeReader $XAML)  
$Form = [Windows.Markup.XamlReader]::Load($Reader)
```

```
$TxtSrc = $Form.FindName("TxtSrc")  
$TxtPaOut = $Form.FindName("TxtPaOut")  
$TxtAxOut = $Form.FindName("TxtAxOut")  
$TxtPaExe = $Form.FindName("TxtPaExe")  
$TxtAxExe = $Form.FindName("TxtAxExe")  
$TxtLog = $Form.FindName("TxtLog")  
$BtnLaunch = $Form.FindName("BtnLaunch")
```

```

function Get-Folder($Title) {
    $FolderBrowser = New-Object System.Windows.Forms.FolderBrowserDialog
    $FolderBrowser.Description = $Title
    if ($FolderBrowser.ShowDialog() -eq "OK") { return $FolderBrowser.SelectedPath }
    return ""
}

$Form.FindName("BtnBrowseSrc").Add_Click({ $TxtSrc.Text = Get-Folder("Select Source .ufd Folder") })
$Form.FindName("BtnBrowsePa").Add_Click({ $TxtPaOut.Text = Get-Folder("Select Cellebrite PA10 Target N
VMe Workspace") })
$Form.FindName("BtnBrowseAx").Add_Click({ $TxtAxOut.Text = Get-Folder("Select Magnet Axiom Target NVMe
Workspace") })

function Write-Log($Message) {
    $Form.Dispatcher.Invoke([Action]{
        $TxtLog.AppendText("[$(Get-Date).ToString('HH:mm:ss')] $Message`r`n")
        $TxtLog.ScrollToEnd()
    })
}

$BtnLaunch.Add_Click({
    if ([string]::IsNullOrEmpty($TxtSrc.Text) -or [string]::IsNullOrEmpty($TxtPaOut.Text) -o
r [string]::IsNullOrEmpty($TxtAxOut.Text)) {
        [System.Windows.MessageBox]::Show("Error: Configure all storage source and output directories.
", "Missing Data", "OK", "Error")
        return
    }
    $BtnLaunch.IsEnabled = $false

    Start-Job -ScriptBlock {
        param($Src, $PaOut, $AxOut, $PaExe, $AxExe)
        function Local-Log($Msg) { $Msg }

        $UfdFiles = Get-ChildItem -Path $Src -Filter "*.ufd"
        if ($UfdFiles.Count -eq 0) {
            Local-Log "[!] Error: No .ufd files detected."
            return
        }
        Local-Log "[+] Detected $($UfdFiles.Count) collections awaiting pipeline processing."

        foreach ($File in $UfdFiles) {
            $BaseName = $File.BaseName
            Local-Log "===== Processing: $BaseName ====="

            $PaItemDir = Join-Path $PaOut "$($BaseName)_PA10_Workspace"
            $AxItemDir = Join-Path $AxOut "$($BaseName)_Axiom_Workspace"
            $null = New-Item -ItemType Directory -Path $PaItemDir -Force
            $null = New-Item -ItemType Directory -Path $AxItemDir -Force

            $CompanionPath = $File.FullName.ToLower().Replace(".ufd", ".bin")
            if (-not (Test-Path $CompanionPath)) { $CompanionPath = $File.FullName.ToLower().Replace("
.ufd", ".tar") }

            Local-Log "[->] Cloning extraction files to isolated tool scratch directories..."
            Copy-Item -Path $File.FullName -Destination (Join-Path $PaItemDir $File.Name) -Force
            Copy-Item -Path $File.FullName -Destination (Join-Path $AxItemDir $File.Name) -Force

            if (Test-Path $CompanionPath) {
                $CompName = Split-Path $CompanionPath -Leaf
                Copy-Item -Path $CompanionPath -Destination (Join-Path $PaItemDir $CompName) -Force
                Copy-Item -Path $CompanionPath -Destination (Join-Path $AxItemDir $CompName) -Force
            }
        }
    }
}

```

```

    }

    Local-Log "[■] Spawning Background Processes..."
    $ProcPA = Start-Process -FilePath $PaExe -ArgumentList "-open `"$PaItemDir\\$(($File.Name))`" -method Forensic -project `"$PaItemDir\\Output`" -examine" -NoNewWindow -PassThru
    $ProcAX = Start-Process -FilePath $AxExe -ArgumentList "/v `"$Mobile`" /i `"$AxItemDir\\$(($File.Name))`" /o `"$AxItemDir\\Output`" /g" -NoNewWindow -PassThru

    while (-not $ProcPA.HasExited -or -not $ProcAX.HasExited) { Start-Sleep -Seconds 5 }
    Local-Log "[✓] Thread execution complete for: $BaseName"
}
Local-Log "■ PIPELINE SUCCESSFUL: All target cases indexed and decoded."
} -ArgumentList $TxtSrc.Text, $TxtPaOut.Text, $TxtAxOut.Text, $TxtPaExe.Text, $TxtAxExe.Text | Out-Null

$Timer = New-Object System.Windows.Threading.DispatcherTimer
$Timer.Interval = [TimeSpan]::FromSeconds(1)
$Timer.Add_Tick({
    $Job = Get-Job | Select-Object -Last 1
    if ($Job) {
        $Data = Receive-Job -Job $Job
        foreach ($Line in $Data) { if ($Line) { Write-Log $Line } }
        if ($Job.State -ne "Running") { $Timer.Stop(); $BtnLaunch.IsEnabled = $True; Remove-Job -Job $Job }
    }
})
$Timer.Start()
})
$Form.ShowDialog() | Out-Null

```

This document outlines advanced local workstation scripting integrations for engineering environments. Ensure security testing protocols match local unit guidelines before execution in secure spaces.